TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
THÀNH PHỐ HỒ CHÍ MINH
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ

BÁO CÁO ĐỀ TÀI

ERD và mô hình quan hệ hệ thống thông tin quản lý khách sạn

Môn: Thiết kế cơ sở dữ liệu

Mã lớp học phần: 012012100211

Sinh viên thực hiện

Họ tên sinh viên	Mã số sinh viên
Hoàng Việt Quang	066206005044
Đoàn Trần Nhật Vy	034306010200
Dương Quỳnh Giang	079306005139
Đàm Triệu Vỹ	066206000409
Nguyễn Duy Nam	066206000017
Hồ Văn Phát	064206001095

Thành phố Hồ Chí Minh, tháng 11 năm 2025

MỤC LỤC

MỤC LỤC ..... **i**
LỜI MỞ ĐẦU ..... **ii**
CHƯƠNG 1: PHÂN TÍCH HỆ THỐNG **1**
1.1. Mô tả bài toán 1
1.2. Các yêu cầu quản lý 1
CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ ERD **4**
2.1. Thực thể và thuộc tính 4
2.2. Các mối quan hệ 6
2.3. Sơ đồ ERD tổng quát 8
Hình 2.1: Sơ đồ ERD tổng quát của hệ thống thông tin quản lý khách sạn 8
CHƯƠNG 3: MÔ HÌNH QUAN HỆ VÀ CÀI ĐẶT **9**
3.1. Mô hình quan hệ 9
3.2. Chuẩn hóa 10
3.3. Cài đặt trên PostgreSQL 11
3.4. Phụ thuộc hàm & Trigger 12
3.4.1. Khái quát về phụ thuộc hàm 12
3.4.2. Ý tưởng Trigger kiểm tra phụ thuộc hàm 13
3.4.3. Cài đặt và mô tả Test Case 13
KẾT LUẬN **15**
TÀI LIỆU THAM KHẢO **16**
PHỤ LỤC **17**
Script cài đặt cơ sở dữ liệu khách sạn: 17
Script phụ thuộc hàm và Trigger: 23

LỜI MỞ ĐẦU

Trong thực tế, hầu hết khách sạn hiện nay đều cần một hệ thống thông tin đủ tin cậy để quản lý khách hàng, phòng, đặt phòng, dịch vụ, nhân viên và hóa đơn một cách tập trung, hạn chế sai sót khi ghi chép thủ công. Xuất phát từ nhu cầu đó, nhóm thực hiện bài tập lớn "Hệ thống thông tin quản lý khách sạn" với mục tiêu thiết kế một cơ sở dữ liệu có cấu trúc rõ ràng, dễ mở rộng và hỗ trợ tốt cho việc khai thác dữ liệu sau này.

Báo cáo tập trung vào việc xây dựng mô hình ERD, chuyển đổi sang mô hình quan hệ và chuẩn hóa dữ liệu (tới ít nhất dạng chuẩn 3NF) cho các thực thể chính như khách hàng, phòng, đơn đặt phòng, dịch vụ, hóa đơn và nhân viên. Trên cơ sở đó, hệ thống được cài đặt trên PostgreSQL với các bảng, ràng buộc toàn vẹn và chỉ mục phù hợp, đồng thời khai thác thêm Function và Trigger để kiểm tra các phụ thuộc hàm được yêu cầu trong đề bài, qua đó tăng tính nhất quán và an toàn cho dữ liệu.

CHƯƠNG 1: PHÂN TÍCH HỆ THỐNG

1.1. Mô tả bài toán

Hệ thống thông tin quản lý khách sạn được xây dựng nhằm hỗ trợ các công việc hằng ngày như quản lý khách hàng, phòng, đặt phòng, dịch vụ, nhân viên và hóa đơn. Thay vì lưu trữ rời rạc trên giấy tờ hoặc nhiều file khác nhau, toàn bộ dữ liệu sẽ được quản lý tập trung trong một cơ sở dữ liệu, giúp tra cứu nhanh hơn, giảm sai sót và thuận tiện khi thống kê, báo cáo.

Khi khách hàng đến đặt phòng hoặc lưu trú, nhân viên tiếp tân ghi nhận thông tin cá nhân và tạo đơn đặt phòng tương ứng, bao gồm các thông tin như phòng được chọn, ngày nhận phòng, ngày trả phòng, số lượng người và tình trạng đơn đặt phòng. Mỗi phòng có thể được sử dụng nhiều lần ở các lượt đặt phòng khác nhau, và trạng thái phòng (có sẵn, đang sử dụng, đang bảo trì, đã đặt) cần được cập nhật liên tục.

Trong thời gian ở tại khách sạn, khách hàng có thể sử dụng thêm nhiều dịch vụ khác nhau như ăn uống, giặt ủi, spa hoặc dịch vụ phòng. Các dịch vụ này được ghi nhận kèm theo từng lần đặt phòng để làm cơ sở tính tiền. Khi khách hàng thanh toán, hệ thống lập hóa đơn cho đơn đặt phòng đó, thể hiện tổng số tiền phải trả, ngày lập hóa đơn, phương thức thanh toán và trạng thái thanh toán. Thông tin về nhân viên và nhiệm vụ phụ trách cũng được lưu trữ để hỗ trợ phân công công việc và theo dõi trách nhiệm trong quá trình vận hành khách sạn.

1.2. Các yêu cầu quản lý

Từ bài toán đặt ra, hệ thống thông tin quản lý khách sạn cần đáp ứng một số nhóm yêu cầu quản lý chính như sau:

- **Quản lý khách hàng**

Hệ thống cho phép lưu trữ và cập nhật đầy đủ thông tin khách hàng như họ tên, địa chỉ, số điện thoại, email và một số thông tin cá nhân khác. Mỗi khi khách có phát sinh đặt phòng, dữ liệu sẽ được gắn với hồ sơ khách hàng tương ứng, giúp theo dõi lịch sử lưu trú và hỗ trợ tra cứu nhanh khi cần.

- **Quản lý phòng**

Hệ thống cần quản lý danh sách phòng với các thông tin cơ bản như mã phòng, loại phòng, đơn giá, tình trạng hiện tại (có sẵn, đang sử dụng, đang bảo trì, đã đặt) và các tiện nghi đi kèm. Mỗi phòng có thể được sử dụng trong nhiều lượt đặt phòng khác nhau theo thời gian, vì vậy trạng thái phòng phải được cập nhật liên tục sau mỗi thao tác đặt, nhận hoặc trả phòng.

- **Quản lý đơn đặt phòng**

Mỗi lần khách đặt phòng sẽ tạo ra một đơn đặt phòng mới, lưu lại thông tin khách hàng, phòng được chọn, ngày đặt, ngày nhận phòng, ngày trả phòng, số người và tình trạng đơn (đã đặt, đã nhận phòng, đã trả phòng, đã hủy). Hệ thống phải đảm bảo một đơn đặt phòng luôn gắn với đúng một khách hàng và một phòng, đồng thời kiểm tra tính hợp lệ của khoảng thời gian nhận – trả phòng.

- **Quản lý dịch vụ**

Khách sạn có thể cung cấp nhiều loại dịch vụ khác nhau như ăn uống, giặt ủi, spa hay dịch vụ phòng. Hệ thống cần quản lý danh mục dịch vụ (tên dịch vụ, đơn giá, đơn vị tính, mô tả) và ghi nhận việc sử dụng dịch vụ theo từng đơn đặt phòng, bao gồm số lượng và đơn giá tại thời điểm sử dụng để làm cơ sở tính tiền.

- **Quản lý hóa đơn**

Sau mỗi lần khách thanh toán, hệ thống lập hóa đơn tương ứng cho một đơn đặt phòng, thể hiện tổng số tiền phải trả (bao gồm tiền phòng và các dịch vụ đã sử dụng), ngày lập

hóa đơn, phương thức thanh toán và trạng thái thanh toán (chưa thanh toán, đã thanh toán, đã hủy). Một đơn đặt phòng có thể phát sinh nhiều hóa đơn khác nhau (ví dụ khách thanh toán nhiều đợt), nhưng mỗi hóa đơn chỉ gắn với một đơn đặt phòng cụ thể.

- **Quản lý nhân viên và nhiệm vụ**

Hệ thống lưu trữ thông tin nhân viên như họ tên, chức vụ, liên hệ, mức lương và ngày vào làm để phục vụ công tác quản lý nhân sự. Bên cạnh đó, mỗi nhân viên có thể được giao nhiều nhiệm vụ khác nhau theo từng thời điểm, vì vậy cần có cách ghi nhận các nhiệm vụ phụ trách để hỗ trợ phân công công việc và đánh giá hiệu quả làm việc.

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ ERD

2.1. Thực thể và thuộc tính

Dựa trên bài toán và sơ đồ ERD, hệ thống thông tin quản lý khách sạn sử dụng các thực thể và thuộc tính chính sau:

- Thực thể Khách hàng (khach_hang)

Mỗi khách hàng được gán một mã duy nhất để phân biệt với các khách khác. Thực thể này lưu trữ các thông tin cơ bản phục vụ liên hệ và tra cứu lịch sử lưu trú.

Thuộc tính: khach_hang_id (PK), ho_ten, so_dien_thoai, email, dia_chi, ngay_sinh, gioi_tinh.

- Thực thể Phòng (phong)

Mỗi phòng trong khách sạn có một mã riêng, kèm theo loại phòng, đơn giá, tình trạng sử dụng và một số thông tin mô tả khác. Đây là cơ sở để gán phòng cho các đơn đặt phòng và tính tiền phòng.

Thuộc tính: phong_id (PK), so_phong (duy nhất), loai_phong, don_gia, tinh_trang, tang, ghi_chu.

- Thực thể Tiện nghi (tien_nghi)

Dùng để quản lý danh sách các tiện nghi có sẵn trong khách sạn (ví dụ: wifi, tivi, điều hòa). Mỗi tiện nghi có mã riêng và mô tả ngắn gọn.

Thuộc tính: tien_nghi_id (PK), ten_tien_nghi, mo_ta.

- Thực thể liên kết Phòng – Tiện nghi (phong_tien_nghi)

Thực thể này dùng để biểu diễn quan hệ nhiều–nhiều giữa Phòng và Tiện nghi: một phòng có thể có nhiều tiện nghi, một tiện nghi có thể xuất hiện ở nhiều phòng.

Thuộc tính: phong_id (PK, FK), tien_nghi_id (PK, FK).

- Thực thể Đơn đặt phòng (dat_phong)

Mỗi lần khách đặt phòng sẽ tạo ra một bản ghi trong bảng này, gắn với đúng một khách hàng và một phòng, kèm theo khoảng thời gian lưu trú và trạng thái đơn.

Thuộc tính: dat_phong_id (PK), khach_hang_id (FK), phong_id (FK), ngay_dat, ngay_nhan, ngay_tra, so_nguoi, trang_thai, ghi_chu.

- Thực thể Nhân viên (nhan_vien)

Lưu trữ thông tin cơ bản về nhân viên khách sạn để phục vụ quản lý nhân sự và phân công công việc.

Thuộc tính: nhan_vien_id (PK), ho_ten, chuc_vu, so_dien_thoai, email, luong, ngay_vao_lam.

- Thực thể Nhiệm vụ (nhiem_vu)

Dùng để mô tả các loại nhiệm vụ mà nhân viên có thể được giao, ví dụ: dọn phòng, lễ tân, bảo trì.

Thuộc tính: nhiem_vu_id (PK), ten_nhiem_vu, mo_ta.

- Thực thể liên kết Nhân viên – Nhiệm vụ (nhan_vien_nhiem_vu)

Biểu diễn quan hệ nhiều–nhiều giữa Nhân viên và Nhiệm vụ, đồng thời ghi lại thời điểm giao nhiệm vụ cho từng nhân viên.

Thuộc tính: nhan_vien_id (PK, FK), nhiem_vu_id (PK, FK), ngay_giao (PK), ghi_chu.

- Thực thể Dịch vụ (dich_vu)

Quản lý danh mục các dịch vụ mà khách sạn cung cấp cho khách trong thời gian lưu trú, cùng với đơn giá và đơn vị tính của từng dịch vụ.

Thuộc tính: dich_vu_id (PK), ten_dich_vu, don_gia, don_vi_tinh, mo_ta.

- Thực thể Sử dụng dịch vụ (su_dung_dich_vu)

Đây là bảng trung gian mô tả việc một đơn đặt phòng cụ thể đã sử dụng những dịch vụ nào, số lượng bao nhiêu và đơn giá tại thời điểm sử dụng. Thực thể này chuyển đổi quan hệ nhiều–nhiều giữa Đơn đặt phòng và Dịch vụ.

Thuộc tính: su_dung_dich_vu_id (PK), dat_phong_id (FK), dich_vu_id (FK), so_luong, don_gia, thoi_gian_su_dung.

- Thực thể Hóa đơn (hoa_don)

Mỗi hóa đơn thể hiện việc thanh toán cho một đơn đặt phòng, bao gồm tổng số tiền phải trả, ngày lập hóa đơn, phương thức thanh toán và trạng thái thanh toán.

Thuộc tính: hoa_don_id (PK), dat_phong_id (FK), khach_hang_id (FK), ngay_lap, tong_tien, phuong_thuc_thanh_toan, trang_thai, ghi_chu.

2.2. Các mối quan hệ

Các thực thể trong hệ thống không tồn tại độc lập mà liên kết với nhau thông qua các mối quan hệ với bội số cụ thể. Dưới đây là các mối quan hệ chính và ý nghĩa quản lý của từng quan hệ:

- Khách hàng – Đơn đặt phòng (1 – N)

Một khách hàng có thể thực hiện nhiều đơn đặt phòng khác nhau theo thời gian (đi nhiều lần, ở nhiều đợt), nhưng mỗi đơn đặt phòng chỉ gắn với một khách hàng cụ thể. Điều này giúp hệ thống theo dõi lịch sử đặt phòng của từng khách và truy vết thông tin khi cần.

- Phòng – Đơn đặt phòng (1 – N)

Mỗi phòng có thể xuất hiện trong nhiều đơn đặt phòng ở các thời điểm khác nhau, tuy nhiên một đơn đặt phòng chỉ sử dụng một phòng tại một thời điểm. Nhờ đó, hệ thống kiểm soát được tần suất sử dụng của từng phòng và hỗ trợ tránh trùng lịch.

- Đơn đặt phòng – Hóa đơn (1 – N)

Một đơn đặt phòng có thể phát sinh nhiều hóa đơn (ví dụ khách thanh toán nhiều đợt hoặc lập hóa đơn bổ sung), nhưng mỗi hóa đơn chỉ liên quan tới đúng một đơn đặt phòng. Quan hệ này giúp việc tổng hợp doanh thu theo từng lần đặt phòng trở nên rõ ràng.

- Nhân viên – Hóa đơn (1 – N)

Mỗi hóa đơn được lập bởi đúng một nhân viên phụ trách, trong khi một nhân viên có thể lập nhiều hóa đơn trong quá trình làm việc. Thông tin này hỗ trợ phân công công việc và đánh giá hiệu quả làm việc của từng nhân viên.

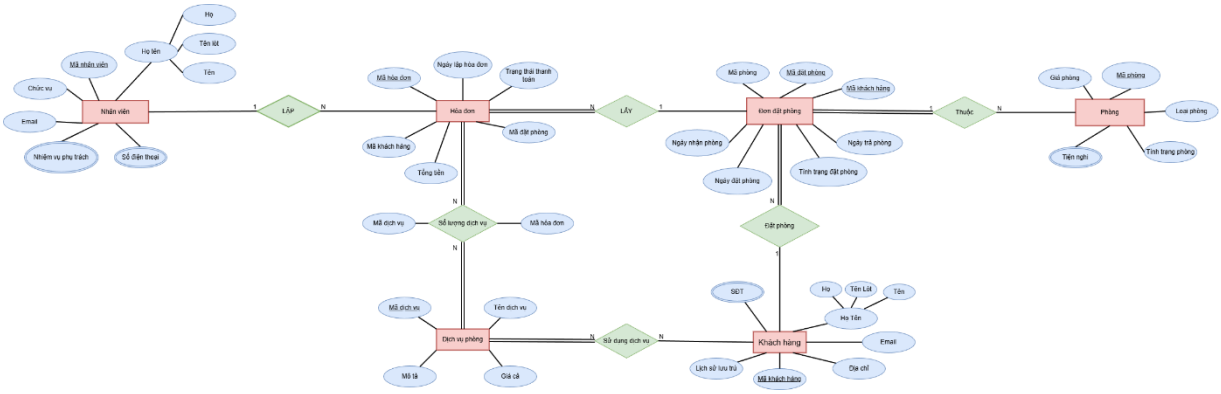
- Dịch vụ – Khách hàng (N – N)

Một khách hàng có thể sử dụng nhiều loại dịch vụ khác nhau trong suốt quá trình lưu trú tại khách sạn, và mỗi loại dịch vụ cũng có thể được nhiều khách hàng khác nhau sử dụng. Mối quan hệ nhiều–nhiều này cho phép thống kê dịch vụ nào được ưa chuộng và phân tích nhóm khách hàng tiềm năng cho từng loại dịch vụ.

- Dịch vụ – Hóa đơn (N – N)

Một hóa đơn có thể bao gồm nhiều loại dịch vụ (ăn uống, giặt ủi, spa, v.v.), và một dịch vụ cũng có thể xuất hiện trên nhiều hóa đơn khác nhau. Quan hệ nhiều–nhiều giữa dịch vụ và hóa đơn phản ánh đúng việc cộng gộp chi phí dịch vụ vào từng lần thanh toán và hỗ trợ phân tích doanh thu theo từng loại dịch vụ.

2.3. Sơ đồ ERD tổng quát



Hình 2.1: Sơ đồ ERD tổng quát của hệ thống thông tin quản lý khách sạn

CHƯƠNG 3: MÔ HÌNH QUAN HỆ VÀ CÀI ĐẶT

3.1. Mô hình quan hệ

Từ sơ đồ ERD ở Chương 2, hệ thống được chuyển đổi sang lược đồ quan hệ để có thể cài đặt trên PostgreSQL. Các quan hệ dưới đây liệt kê đầy đủ các thuộc tính, trong đó (PK) là khóa chính và (FK) là khóa ngoại.

1. khach_hang (khach_hang_id (PK), ho_ten, so_dien_thoai, email, dia_chi, ngay_sinh, gioi_tinh)

2. phong (phong_id (PK), so_phong, loai_phong, don_gia, tinh_trang, tang, ghi_chu)

3. tien_nghi (tien_nghi_id (PK), ten_tien_nghi, mo_ta)

4. phong_tien_nghi (phong_id (PK, FK), tien_nghi_id (PK, FK))

   - phong_id tham chiếu đến bảng phong

   - tien_nghi_id tham chiếu đến bảng tien_nghi

5. dat_phong (dat_phong_id (PK), khach_hang_id (FK), phong_id (FK), ngay_dat, ngay_nhan, ngay_tra, so_nguoi, trang_thai, ghi_chu)

   - khach_hang_id tham chiếu đến bảng khach_hang

   - phong_id tham chiếu đến bảng phong

6. nhan_vien (nhan_vien_id (PK), ho_ten, chuc_vu, so_dien_thoai, email, luong, ngay_vao_lam)

7. nhiem_vu (nhiem_vu_id (PK), ten_nhiem_vu, mo_ta)

8. nhan_vien_nhiem_vu (nhan_vien_id (PK, FK), nhiem_vu_id (PK, FK), ngay_giao (PK), ghi_chu)

   - nhan_vien_id tham chiếu đến bảng nhan_vien

- nhiem_vu_id tham chiếu đến bảng nhiem_vu

9. dich_vu (dich_vu_id (PK), ten_dich_vu, don_gia, don_vi_tinh, mo_ta)

10. su_dung_dich_vu (su_dung_dich_vu_id (PK), dat_phong_id (FK), dich_vu_id (FK), so_luong, don_gia, thoi_gian_su_dung)

- dat_phong_id tham chiếu đến bảng dat_phong

- dich_vu_id tham chiếu đến bảng dich_vu

11. hoa_don (hoa_don_id (PK), dat_phong_id (FK), khach_hang_id (FK), ngay_lap, tong_tien, phuong_thuc_thanh_toan, trang_thai, ghi_chu)

- dat_phong_id tham chiếu đến bảng dat_phong

- khach_hang_id tham chiếu đến bảng khach_hang

Các quan hệ này đảm bảo bám sát yêu cầu nghiệp vụ: quản lý khách hàng, phòng, đặt phòng, dịch vụ, hóa đơn và nhân viên, đồng thời hỗ trợ tốt cho việc triển khai các ràng buộc toàn vẹn và tối ưu truy vấn trong phần cài đặt SQL.

3.2. Chuẩn hóa

Các quan hệ trong cơ sở dữ liệu được thiết kế để đạt tới dạng chuẩn 3 (3NF), nhằm hạn chế trùng lặp dữ liệu và giảm nguy cơ sai lệch khi cập nhật. Quá trình chuẩn hóa được thực hiện lần lượt qua các mức 1NF, 2NF và 3NF như sau [1]:

- Dạng chuẩn 1 (1NF) – Giá trị nguyên tố

Tất cả các thuộc tính trong bảng đều là giá trị đơn, không lưu danh sách hay nhóm lặp trong cùng một ô [2]. Ví dụ, thay vì ghi nhiều tiện nghi của một phòng trong một cột (như “TV, Tủ lạnh, Máy lạnh”), hệ thống tách danh sách tiện nghi ra bảng tien_nghi và sử dụng bảng trung gian phong_tien_nghi để liên kết phòng với từng tiện nghi riêng lẻ.

- Dạng chuẩn 2 (2NF) – Phụ thuộc đầy đủ vào khóa chính

Với các bảng có khóa chính là tổ hợp nhiều thuộc tính (như phong_tien_nghi hay nhan_vien_nhiem_vu), mọi thuộc tính phi khóa đều phụ thuộc vào toàn bộ khóa, không phụ thuộc vào từng phần của khóa [3]. Chẳng hạn, trong bảng su_dung_dich_vu, các thuộc tính so_luong và don_gia gắn với từng lần sử dụng dịch vụ (su_dung_dich_vu_id), không phụ thuộc riêng lẻ chỉ vào dat_phong_id hay dich_vu_id.

- Dạng chuẩn 3 (3NF) – Loại bỏ phụ thuộc bắc cầu

Không có thuộc tính phi khóa nào phụ thuộc vào một thuộc tính phi khóa khác [2]. Ví dụ, trong bảng dat_phong chỉ lưu phong_id, còn thông tin don_gia của phòng được giữ ở bảng phong. Nếu vừa lưu phong_id vừa lặp lại don_gia trong dat_phong thì khi thay đổi giá phòng sẽ phải cập nhật nhiều nơi, dễ gây lệch dữ liệu. Cách tách như hiện tại giúp dữ liệu gọn hơn và nhất quán hơn.

3.3. Cài đặt trên PostgreSQL

Dựa trên mô hình quan hệ ở mục 3.1, nhóm tiến hành cài đặt cơ sở dữ liệu trên hệ quản trị PostgreSQL bằng các script SQL [4], [5]. Mỗi thực thể trong mô hình được ánh xạ thành một bảng, kèm theo các ràng buộc khóa chính, khóa ngoại và các CHECK cần thiết để đảm bảo dữ liệu nhập vào luôn hợp lệ.

Về kiểu dữ liệu, các khóa chính như khach_hang_id, phong_id, dat_phong_id, hoa_don_id,… đều sử dụng kiểu SERIAL để tự động tăng, giúp việc quản lý mã định danh đơn giản hơn. Các thuộc tính văn bản dùng VARCHAR với độ dài phù hợp (ví dụ ho_ten, email, so_dien_thoai), còn các giá trị tiền tệ như don_gia, tong_tien dùng kiểu NUMERIC với số lẻ cố định để tránh sai số khi tính toán.

Hệ thống cũng sử dụng khá nhiều ràng buộc CHECK để kiểm soát dữ liệu đầu vào [4], chẳng hạn kiểm tra định dạng email bằng biểu thức chính quy, giới hạn tập giá trị cho

các cột gioi_tinh, tinh_trang phòng, trang_thai đơn đặt phòng và trạng_thai hóa đơn. Một số ràng buộc khác kiểm tra tính hợp lệ về thời gian (ngày nhận phòng không trước ngày đặt, ngày trả phòng không trước ngày nhận) và đảm bảo các giá trị như so_nguoi, don_gia, luong luôn lớn hơn 0.

Ngoài ra, các chỉ mục (INDEX) được tạo trên những cột thường xuyên dùng để tìm kiếm và liên kết bảng như khach_hang_id, phong_id, dat_phong_id, so_phong, email, so_dien_thoai và trang_thai. Việc này giúp các câu lệnh SELECT, JOIN và các truy vấn thống kê hoạt động nhanh hơn trên tập dữ liệu lớn, đồng thời vẫn giữ nguyên cấu trúc chuẩn hóa của cơ sở dữ liệu.

3.4. Phụ thuộc hàm & Trigger

3.4.1. Khái quát về phụ thuộc hàm

Trong mô hình dữ liệu quan hệ, phụ thuộc hàm là ràng buộc thể hiện việc một tập thuộc tính xác định duy nhất một tập thuộc tính khác [3]. Cho quan hệ R và hai tập thuộc tính X, Y ⊆ R, ta nói X xác định hàm Y (kí hiệu X → Y) nếu với mỗi giá trị của X chỉ có đúng một giá trị tương ứng của Y. Phụ thuộc hàm là cơ sở lý thuyết quan trọng cho việc chuẩn hóa lược đồ và đảm bảo tính nhất quán của dữ liệu.

Trong phạm vi này xem xét hai phụ thuộc hàm điển hình:

- Trên quan hệ R1(A, B, C): phụ thuộc A → B, nghĩa là mỗi giá trị A chỉ được gắn với một giá trị B duy nhất [3].

- Trên quan hệ R2(A, B, C, D, E): phụ thuộc AB → CD, nghĩa là mỗi cặp (A, B) chỉ được gắn với một cặp (C, D) duy nhất [3].

3.4.2. Ý tưởng Trigger kiểm tra phụ thuộc hàm

Đối với những phụ thuộc hàm không thể biểu diễn trực tiếp bằng khóa chính hoặc ràng buộc UNIQUE, có thể sử dụng trigger để kiểm tra ràng buộc trước khi dữ liệu được ghi vào bảng [6].

Giải thuật Trigger (BEFORE INSERT/UPDATE) cho phụ thuộc dạng A → B có thể mô tả như sau:

1. Khi chèn mới hoặc cập nhật một bản ghi với giá trị A = a, B = b.

2. Trigger kiểm tra toàn bộ các bản ghi hiện có trong bảng.

3. Nếu tồn tại bản ghi khác có cùng giá trị A = a nhưng B khác b thì:

– Phát sinh lỗi (RAISE EXCEPTION) và hủy thao tác chèn/cập nhật vì vi phạm phụ thuộc hàm A → B.

4. Nếu không phát hiện bản ghi vi phạm thì cho phép thực hiện thao tác chèn/cập nhật dữ liệu.

Đối với phụ thuộc AB → CD, ý tưởng tương tự, nhưng việc so sánh được thực hiện trên cặp (A, B) và cặp (C, D) thay vì một thuộc tính đơn.

3.4.3. Cài đặt và mô tả Test Case

Trên PostgreSQL, mỗi phụ thuộc hàm được hiện thực bằng một hàm PL/pgSQL (trigger function) và một trigger gắn với bảng tương ứng [6], [7]:

- Với phụ thuộc A → B trên quan hệ R1(A, B, C):

+ Định nghĩa hàm fd_r1_a_to_b() kiểm tra xem đã tồn tại bản ghi có cùng A nhưng B khác hay chưa.

+ Gắn hàm này với bảng R1 thông qua trigger BEFORE INSERT OR UPDATE FOR EACH ROW.

- Với phụ thuộc AB → CD trên quan hệ R2(A, B, C, D, E):

+ Định nghĩa hàm fd_r2_ab_to_cd() kiểm tra xem đã tồn tại bản ghi có cùng (A, B) nhưng (C, D) khác hay chưa.

+ Gắn hàm này với bảng R2 bằng trigger BEFORE INSERT OR UPDATE FOR EACH ROW.

Mô tả Test Case minh họa cho phụ thuộc A → B:

- Dữ liệu ban đầu: (A = 1, B = 100).

- Test case 1 (hợp lệ): chèn thêm bản ghi (A = 2, B = 200) hoặc (A = 1, B = 100, C khác) → thao tác được chấp nhận vì không thay đổi giá trị B ứng với A.

- Test case 2 (vi phạm): chèn bản ghi (A = 1, B = 500) → trigger phát sinh lỗi vì A = 1 đã tồn tại với giá trị B khác, vi phạm phụ thuộc A → B.

Các test case tương tự có thể được xây dựng cho phụ thuộc AB → CD trên R2 bằng cách giữ nguyên cặp (A, B) và thay đổi cặp (C, D) để kiểm tra hoạt động của trigger.

KẾT LUẬN

Đề tài đã xây dựng được mô hình cơ sở dữ liệu cho hệ thống thông tin quản lý khách sạn theo đúng quy trình: phân tích yêu cầu, thiết kế mô hình ERD, chuyển đổi sang mô hình quan hệ và chuẩn hóa đến dạng chuẩn 3NF [1]. Các thực thể chính như khách hàng, phòng, đơn đặt phòng, hóa đơn, dịch vụ và nhân viên được tổ chức lại một cách logic, hạn chế trùng lặp dữ liệu và hỗ trợ tốt cho việc mở rộng hệ thống sau này.

Trên cơ sở mô hình đã thiết kế, cơ sở dữ liệu được cài đặt trên PostgreSQL với đầy đủ các ràng buộc toàn vẹn như khóa chính, khóa ngoại, ràng buộc CHECK, UNIQUE và các chỉ mục phục vụ truy vấn [4]. Việc kết hợp phụ thuộc hàm với Trigger ở phần cuối giúp mô phỏng các ràng buộc nghiệp vụ phức tạp mà các constraint thông thường chưa biểu diễn hết, qua đó tăng mức độ an toàn và nhất quán cho dữ liệu [4], [6].

Mặc dù hệ thống mới dừng lại ở mức mô hình dữ liệu và các lệnh SQL cơ bản, kết quả đạt được cho thấy cách tiếp cận thiết kế bài bản mang lại một lược đồ rõ ràng, dễ bảo trì và có thể làm nền tảng để phát triển tiếp các chức năng ứng dụng như giao diện đặt phòng, module báo cáo doanh thu hay các tính năng phân tích dữ liệu trong tương lai.

TÀI LIỆU THAM KHẢO

[1] IBM, “What Is Database Normalization?” [Online]. Available: https://www.ibm.com/think/topics/database-normalization

[2] GeeksforGeeks, “Partial, Full, and Transitive Dependencies.” [Online]. Available: https://www.geeksforgeeks.org/dbms/partial-full-and-transitive-dependencies/

[3] GeeksforGeeks, “Fully Functional Dependency in DBMS.” [Online]. Available: https://www.geeksforgeeks.org/dbms/fully-functional-dependency-in-dbms/

[4] PostgreSQL Global Development Group, “Chapter 5. Data Definition – 5.5. Constraints.” [Online]. Available: https://www.postgresql.org/docs/current/ddl-constraints.html

[5] PostgreSQL Global Development Group, “CREATE TABLE.” [Online]. Available: https://www.postgresql.org/docs/current/sql-createtable.html

[6] CYBERTEC PostgreSQL International GmbH, “Triggers to Enforce Constraints in PostgreSQL.” [Online]. Available: https://www.cybertec-postgresql.com/en/triggers-to-enforce-constraints/

[7] DataCamp, “PostgreSQL BEFORE Triggers.” [Online]. Available: https://www.datacamp.com/doc/postgresql/before-triggers

PHỤ LỤC

Script cài đặt cơ sở dữ liệu khách sạn:

```
CREATE TABLE khach_hang (
    khach_hang_id SERIAL PRIMARY KEY,
    ho_ten VARCHAR(100) NOT NULL,
    so_dien_thoai VARCHAR(20),
    email VARCHAR(100),
    dia_chi TEXT,
    ngay_sinh DATE,
    gioi_tinh VARCHAR(10) CHECK (gioi_tinh IN ('Nam', 'Nữ', 'Khác') OR gioi_tinh IS NULL),
    CONSTRAINT chk_email_format CHECK (email IS NULL OR email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
);

CREATE TABLE phong (
    phong_id SERIAL PRIMARY KEY,
    so_phong VARCHAR(10) UNIQUE NOT NULL,
    loai_phong VARCHAR(50) NOT NULL,
    don_gia NUMERIC(12,2) NOT NULL CHECK (don_gia >= 0),
```

```
    tinh_trang VARCHAR(20) NOT NULL CHECK (tinh_trang IN ('Có sẵn', 'Đang sử dụng', 'Đang bảo trì', 'Đã đặt')),
    tang INT CHECK (tang > 0),
    ghi_chu TEXT
);

CREATE TABLE tien_nghi (
    tien_nghi_id SERIAL PRIMARY KEY,
    ten_tien_nghi VARCHAR(100) NOT NULL,
    mo_ta TEXT
);

CREATE TABLE phong_tien_nghi (
    phong_id INT REFERENCES phong(phong_id) ON DELETE CASCADE,
    tien_nghi_id INT REFERENCES tien_nghi(tien_nghi_id) ON DELETE CASCADE,
    PRIMARY KEY (phong_id, tien_nghi_id)
);

CREATE TABLE dat_phong (
    dat_phong_id SERIAL PRIMARY KEY,
```

```
    khach_hang_id INT REFERENCES khach_hang(khach_hang_id) ON DELETE RESTRICT NOT NULL,
    phong_id INT REFERENCES phong(phong_id) ON DELETE RESTRICT NOT NULL,
    ngay_dat DATE NOT NULL DEFAULT CURRENT_DATE,
    ngay_nhan DATE,
    ngay_tra DATE,
    so_nguoi INT CHECK (so_nguoi > 0),
    trang_thai VARCHAR(30) NOT NULL CHECK (trang_thai IN ('Đã đặt', 'Đã nhận phòng', 'Đã trả phòng', 'Đã hủy')),
    ghi_chu TEXT,
    CONSTRAINT chk_ngay_dat_phong CHECK (
        (ngay_nhan IS NULL OR ngay_nhan >= ngay_dat) AND
        (ngay_tra IS NULL OR ngay_nhan IS NULL OR ngay_tra >= ngay_nhan)
    )
);

CREATE TABLE nhan_vien (
    nhan_vien_id SERIAL PRIMARY KEY,
    ho_ten VARCHAR(100) NOT NULL,
    chuc_vu VARCHAR(50),
```

```
    so_dien_thoai VARCHAR(20),
    email VARCHAR(100),
    luong NUMERIC(12,2) CHECK (luong >= 0),
    ngay_vao_lam DATE,
    CONSTRAINT chk_nhan_vien_email_format CHECK (email IS NULL OR email
~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
);

CREATE TABLE nhiem_vu (
    nhiem_vu_id SERIAL PRIMARY KEY,
    ten_nhiem_vu VARCHAR(100) NOT NULL,
    mo_ta TEXT
);

CREATE TABLE nhan_vien_nhiem_vu (
    nhan_vien_id INT REFERENCES nhan_vien(nhan_vien_id) ON DELETE
CASCADE,
    nhiem_vu_id INT REFERENCES nhiem_vu(nhiem_vu_id) ON DELETE
CASCADE,
    ngay_giao DATE,
    ghi_chu TEXT,
```

```
    PRIMARY KEY (nhan_vien_id, nhiem_vu_id, ngay_giao)
);

CREATE TABLE dich_vu (
    dich_vu_id SERIAL PRIMARY KEY,
    ten_dich_vu VARCHAR(100) NOT NULL,
    don_gia NUMERIC(12,2) NOT NULL CHECK (don_gia >= 0),
    don_vi_tinh VARCHAR(20),
    mo_ta TEXT
);

CREATE TABLE su_dung_dich_vu (
    su_dung_dich_vu_id SERIAL PRIMARY KEY,
    dat_phong_id INT REFERENCES dat_phong(dat_phong_id) ON DELETE
CASCADE NOT NULL,
    dich_vu_id INT REFERENCES dich_vu(dich_vu_id) ON DELETE RESTRICT
NOT NULL,
    so_luong INT NOT NULL CHECK (so_luong > 0),
    don_gia NUMERIC(12,2) NOT NULL CHECK (don_gia >= 0),
    thoi_gian_su_dung TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE hoa_don (
    hoa_don_id SERIAL PRIMARY KEY,
    dat_phong_id INT REFERENCES dat_phong(dat_phong_id) ON DELETE
RESTRICT NOT NULL,
    khach_hang_id INT REFERENCES khach_hang(khach_hang_id) ON DELETE
RESTRICT NOT NULL,
    ngay_lap DATE NOT NULL DEFAULT CURRENT_DATE,
    tong_tien NUMERIC(14,2) NOT NULL CHECK (tong_tien >= 0),
    phuong_thuc_thanh_toan VARCHAR(30),
    trang_thai VARCHAR(30) NOT NULL CHECK (trang_thai IN ('Chưa thanh toán',
'Đã thanh toán', 'Đã hủy')),
    ghi_chu TEXT
);


CREATE INDEX idx_dat_phong_khach_hang ON dat_phong(khach_hang_id);
CREATE INDEX idx_dat_phong_phong ON dat_phong(phong_id);
CREATE INDEX idx_su_dung_dich_vu_dat_phong ON
su_dung_dich_vu(dat_phong_id);
CREATE INDEX idx_su_dung_dich_vu_dich_vu ON su_dung_dich_vu(dich_vu_id);
CREATE INDEX idx_hoa_don_dat_phong ON hoa_don(dat_phong_id);
```

```
CREATE INDEX idx_hoa_don_khach_hang ON hoa_don(khach_hang_id);

CREATE INDEX idx_nhan_vien_nhiem_vu_nhan_vien ON
nhan_vien_nhiem_vu(nhan_vien_id);

CREATE INDEX idx_nhan_vien_nhiem_vu_nhiem_vu ON
nhan_vien_nhiem_vu(nhiem_vu_id);

CREATE INDEX idx_phong_tien_nghi_phong ON phong_tien_nghi(phong_id);

CREATE INDEX idx_phong_tien_nghi_tien_nghi ON phong_tien_nghi(tien_nghi_id);


CREATE INDEX idx_khach_hang_email ON khach_hang(email) WHERE email IS
NOT NULL;

CREATE INDEX idx_khach_hang_so_dien_thoai ON khach_hang(so_dien_thoai)
WHERE so_dien_thoai IS NOT NULL;

CREATE INDEX idx_phong_so_phong ON phong(so_phong);

CREATE INDEX idx_phong_tinh_trang ON phong(tinh_trang);

CREATE INDEX idx_dat_phong_trang_thai ON dat_phong(trang_thai);

CREATE INDEX idx_dat_phong_ngay_nhan ON dat_phong(ngay_nhan);

CREATE INDEX idx_hoa_don_trang_thai ON hoa_don(trang_thai);

CREATE INDEX idx_hoa_don_ngay_lap ON hoa_don(ngay_lap);
```

Script phụ thuộc hàm và Trigger:

```
DROP TABLE IF EXISTS r1 CASCADE;
```

```
CREATE TABLE r1 (
   a INT NOT NULL,
   b INT NOT NULL,
   c INT,
   PRIMARY KEY (a, c)
);

DROP TABLE IF EXISTS r2 CASCADE;
CREATE TABLE r2 (
   a INT NOT NULL,
   b INT NOT NULL,
   c INT NOT NULL,
   d INT NOT NULL,
   e INT,
   PRIMARY KEY (a, b, e)
);

CREATE OR REPLACE FUNCTION fd_r1_a_to_b()
RETURNS TRIGGER
LANGUAGE plpgsql
```

```
AS $$
BEGIN
   IF EXISTS (
      SELECT 1
      FROM r1
      WHERE a = NEW.a
        AND b IS DISTINCT FROM NEW.b
        AND (TG_OP = 'INSERT' OR (a, b, c) IS DISTINCT FROM (OLD.a, OLD.b, OLD.c))
   ) THEN
      RAISE EXCEPTION 'Vi pham phu thuoc ham A -> B tren R1: Gia tri a=% da ton tai voi b khac',
          NEW.a;
   END IF;
   RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_fd_r1_a_to_b ON r1;
CREATE TRIGGER trg_fd_r1_a_to_b
BEFORE INSERT OR UPDATE ON r1
```

```
FOR EACH ROW
EXECUTE FUNCTION fd_r1_a_to_b();


CREATE OR REPLACE FUNCTION fd_r2_ab_to_cd()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
   IF EXISTS (
      SELECT 1
      FROM r2
      WHERE a = NEW.a
        AND b = NEW.b
        AND (c IS DISTINCT FROM NEW.c OR d IS DISTINCT FROM NEW.d)
        AND (TG_OP = 'INSERT' OR (a, b, c, d, e) IS DISTINCT FROM (OLD.a,
OLD.b, OLD.c, OLD.d, OLD.e))
   ) THEN
      RAISE EXCEPTION 'Vi pham phu thuoc ham AB -> CD tren R2: Cap gia tri
(a=%, b=%) da ton tai voi (c, d) khac',
          NEW.a, NEW.b;
   END IF;
```

```
    RETURN NEW;
END;
$$;


DROP TRIGGER IF EXISTS trg_fd_r2_ab_to_cd ON r2;
CREATE TRIGGER trg_fd_r2_ab_to_cd
BEFORE INSERT OR UPDATE ON r2
FOR EACH ROW
EXECUTE FUNCTION fd_r2_ab_to_cd();
```